invoi

TECHNICAL DOCUMENTATION

How invoi is built, run and extended

Architecture, data model, API, frontend, AI extraction, automations, infrastructure and roadmap, in one reference synced with the code. This edition records the AI extraction and analytics designs and the decisions behind them.

استخرج بيانات الفواتير بسهولة وبدون مجهود

v1.3 · 22 October 2026 Source: JxTechStudio-org/invoi @ develop (PR #113) JX Tech Studio

CONTENTS

Inside this document

LIVE
invoi.jxtechstudio.com

REPOSITORY
JxTechStudio-org/invoi · develop @ 07f2e22

STACK
React 19 · NestJS 11 · PostgreSQL 16 · RustFS

STATUS
Platform built · extraction and analytics designed

01 **Overview** 3
What invoi is, who it serves and what works today

02 **Architecture** 4
How the services connect on the production server

03 **Tech stack and repository** 5
Libraries, versions and where the code lives

04 **Invoice lifecycle** 7
From upload through extraction to completed

05 **Data model** 8
Tables, fields, enums and planned schema changes

06 **API reference** 10
Every endpoint and the rules behind it

07 **Frontend** 12
Routes, data flow and the design system

08 **AI extraction: Phase 1 design** 14
Pipeline, confidence checks, review rules, failure and rollout

09 **Decision log** 17
38 decisions from 21 Oct 2026, B6, B8 and C13 amended 22 Oct; what is still open

10 **Analytics, n8n and Power BI** 19
Analytics page, n8n webhooks, scheduled reports, Power BI templates

11 **Infrastructure and deployment** 21
Docker, Jenkins, environment and monitoring

12 **Developer guide** 23
Local setup, scripts, Git workflow, conventions

13 **Status, known gaps and roadmap** 25
Closed and open gaps, Phase 1 work and the roadmap

14 **Related documents and changelog** 27
Notion pages, brand kit and the changelog

1 Overview

invoi is an AI-powered invoice management dashboard for small Saudi businesses: users upload incoming invoices, invoi stores and organises them, and flags the ones that need a human check. This document describes the system as it is on the `develop` branch of JxTechStudio-org/invoi (last merge 22 Oct 2026, PR #113). `main` was brought level with `develop` the same day (PR #114), so everything described as built here is on the production branch.

Problem. Businesses that receive many invoices a day (100+ from different vendors and payment methods) re-type vendor, amount, date and tax by hand. It is slow and error-prone.

Users. Small business owners and accountants without a dedicated finance team.

Scope of this version. Incoming invoices only (expenses the user receives from vendors). The customer on every invoice is therefore always the account owner, identified by `User.businessName`. Arabic and English invoices are both first-class.

In scope (MVP)	Out of scope for now
Upload, read, edit, approve and delete invoices	Outgoing invoices the user issues
Per-account storage, one user per account	Multi-user accounts and teams
AI extraction of 12 fields (ZATCA QR first)	ERP or accounting-system integrations
Rule-based flagging for manual review	Line-item (SKU) detail, credit notes, multi-file upload

Live: invoi.jxtechstudio.com (landing page, app and API on one domain; API under `/api`).

Team

Person	Area
Jana	Systems and infrastructure, product owner; prepares the extraction test set
Ahmed	Backend (NestJS, database, storage, CI)
Rahaf	Full-stack: invoice list, upload, notifications
Danah	Full-stack: invoice detail, auth, analytics page, landing page
Abdullah	Power BI reporting and n8n workflows

What works today: sign-up and login (JWT), upload of PDF/JPEG/PNG to object storage, invoice list with search, status and date filters and CSV export, invoice detail with a preview of the original file, manual editing of all 13 editable fields, approve and report-error actions, delete with confirmation, the review rules, dashboard and vendors pages, health and Prometheus metrics, and a Jenkins pipeline that tests and deploys on every build.

What does not exist yet: the AI extraction itself. Uploaded invoices stay in `processing` with empty fields until someone fills them in by hand. Section 8 is the agreed extraction design, section 10 the analytics and n8n design, and section 9 records every decision behind both.

2 Architecture

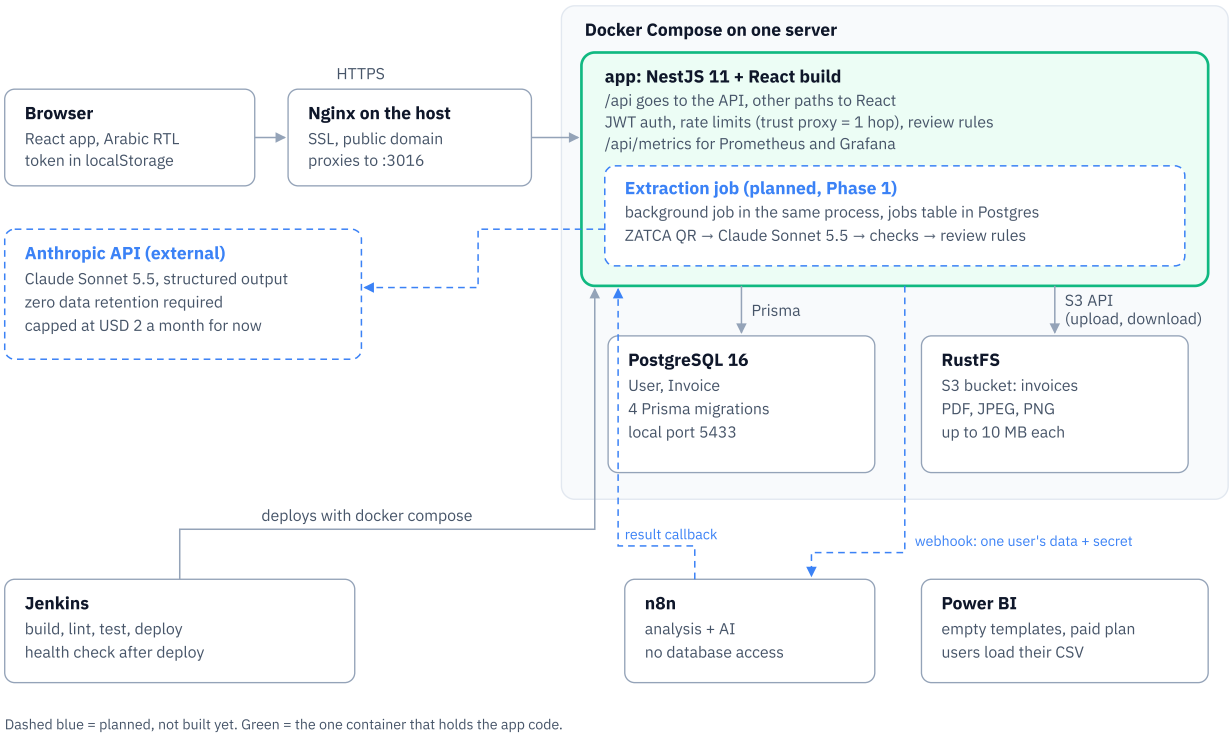


Figure 1 · invoi architecture on the production server, with the planned extraction job and n8n webhooks

One container serves both the API and the web app; PostgreSQL holds invoice data and RustFS holds the original files. n8n sits outside the app and only talks to it through webhooks: the app sends one user's data, n8n calls back with the result (section 10). Power BI becomes downloadable templates and no longer reads the database. Extraction will run inside the same container as a background job (decision C10); the only new outside dependency is the Anthropic API.

3 Tech stack and repository

One repository holds both apps. They ship as a single Docker image: NestJS serves the API under `/api` and the built React app for every other path.

Layer	Technology	Version (package.json)
Frontend	React, TypeScript, Vite	React 19.2, Vite 8.2, TS 7.0
UI kit	Ant Design, framer-motion, Recharts	antd 6.6, Recharts 3.10
Data fetching	TanStack Query, Axios	Query 5.102, Axios 1.20
Routing	React Router	7.18
Backend	NestJS, TypeScript	Nest 11, TS 5.7
ORM and database	Prisma, PostgreSQL	Prisma 6.6, Postgres 16
Auth	Passport JWT, bcrypt	passport-jwt 4, bcrypt 6
Object storage	RustFS (S3-compatible) via AWS SDK v3	rustfs 1.0.0-rc.6
Rate limiting	@nestjs/throttler	60 req/min default, 10/min on login and register
Metrics	prom-client	`invoi_http_requests_total` + Node defaults
AI extraction	Anthropic API (planned)	Claude Sonnet 5.5, `claude-sonnet-5-5`
Runtime	Node.js	22 (alpine)
CI/CD	Jenkins, Docker Compose	Build, Lint, Test, Deploy

Repository layout

```
invoi/
├── backend/
│   ├── prisma/            schema.prisma + 4 migrations
│   ├── src/
│   │   ├── auth/          register, login, me, JWT guard
│   │   ├── invoices/      controller, service, review rules, CSV export
│   │   ├── storage/       RustFS + legacy local storage: upload, download, remove
│   │   ├── common/        global exception filter (logs 5xx with stack)
│   │   ├── metrics/       Prometheus endpoint + HTTP interceptor
│   │   ├── health/        GET /api/health
│   │   ├── proxy.config.ts trust exactly one proxy hop
│   │   └── main.ts        /api prefix, CORS, validation, serves frontend/dist
│   └── test/              15 e2e specs (Jest)
├── frontend/
│   └── src/
│       ├── features/      auth, dashboard, invoice, vendors, landing, shared
│       ├── services/api/  Axios client, React Query client
│       └── config/theme/  antd light and dark themes
├── docs/
│   └── technical-documentation/ this document: source + build script
├── Dockerfile             3-stage build (frontend, backend, runtime)
├── docker-compose.yml     app, postgres, rustfs
├── Jenkinsfile
└── .env.example
```

Phase 1 adds `backend/src/extraction/` (job runner, ZATCA QR reader, model client, checks). The root README still lists a `reports/` folder for Power BI assets; it is not in the repository.

4 Invoice lifecycle

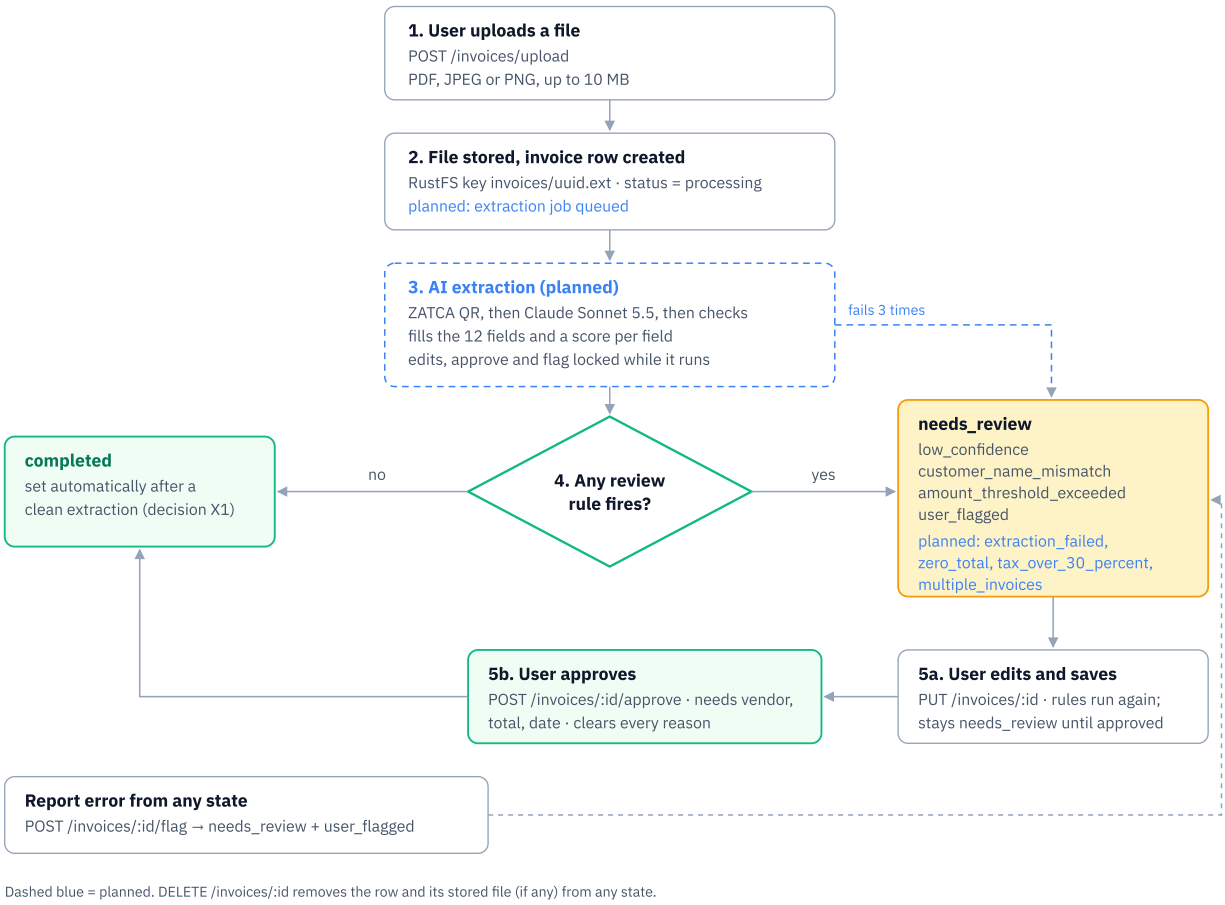


Figure 2 · invoice lifecycle and its three statuses

Today steps 1, 2, 4 and 5 run. Step 3 is missing, so the rules only fire when a user saves an edit, and a manually filled invoice reaches `completed` only through approve. With extraction, a clean result goes straight to `completed`; anything flagged waits for a human. The three statuses stay `processing`, `needs_review` and `completed`; no new status is needed.

5 Data model

Two tables in PostgreSQL, one user to many invoices. IDs are Prisma `cuid` strings and money is `DECIMAL(12,2)`. Source: `backend/prisma/schema.prisma`, 4 migrations from 8 to 22 Sep 2026.

User

Field	Type	Notes
id	string	Primary key, cuid
email	string	Unique; login identifier
passwordHash	string	bcrypt, cost 10
businessName	string	The customer identity for every invoice in the account
firstName, lastName	string	Required; Arabic or English letters
username	string	Required; not unique
phone	string	Required; format `+9665XXXXXXXX`; not unique
reviewAmountThreshold	decimal?	Invoices above this amount are flagged; settable from the settings page in Phase 1
createdAt	datetime	Default now

Invoice

Field	Type	Notes
id	string	Primary key, cuid
userId	string	FK to User.id, ON DELETE RESTRICT; indexed
fileUrl	string	`s3://invoices/invoices/<uuid>.<ext>` (RustFS), `/uploads/<file>` (legacy) or `seed://…` (demo data, no stored file); served by `GET /invoices/:id/file`
status	InvoiceStatus	Default `processing`; indexed
vendorName, invoiceNumber	string?	Extracted
invoiceDate, dueDate	datetime?	`invoiceDate` indexed
totalAmount, taxAmount, amountPaid	decimal?	12,2
currency	Currency?	
paymentStatus, paymentMethod	enum?	Extracted only when clearly printed (decision F23)
customerName	string?	Name as printed on the invoice; compared with `businessName`
taxNumber, crNumber	string?	Vendor VAT number and commercial registration
extractionConfidence	json?	Per-field scores 0–1, e.g. `{"totalAmount": 0.82}`, computed by code checks
needsReviewReason	string?	Comma-separated reason codes (section 8)
createdAt, updatedAt	datetime	Automatic

Enums

Enum	Values
InvoiceStatus	processing, needs_review, completed (unchanged in Phase 1)
PaymentStatus	paid, unpaid, partially_paid, overdue
PaymentMethod	bank_transfer, credit_card, cash, cheque, online_payment, mixed
Currency	SAR, USD, AED · Phase 1 + EUR, GBP, KWD, QAR, BHD, OMR, EGP, JOD, CNY, INR

API responses also carry `sellerName` and `amount`, copies of `vendorName` and `totalAmount` kept for older frontend code.

Planned schema changes (one Phase 1 migration)

Change	Type	Why
`User.businessNameEn`	string?	English business name; the customer-name check compares against both names (F19)
`Currency` + 10 values	enum	Vendors bill in other currencies (F22)
`Invoice.editedFields`	string[]	Fields the user changed by hand; re-extraction never overwrites them (C12)
`Invoice.extractionModel`	string?	Model ID used, e.g. `claude-sonnet-5-5`, or `zatca_qr` only (E17)
`Invoice.extractionDurationMs`	int?	Time taken (E17)
`Invoice.extractionCostUsd`	decimal(10,6)?	Cost of the call; summed for the monthly budget guard (A4, E17)
`Invoice.extractedAt`	datetime?	When the last extraction finished
`Invoice.paidAt`	datetime?	Real payment date for DPO; set when `paymentStatus` becomes `paid`, editable. Demo data copied once from `updatedAt` (H7)
`User.isDemo`	boolean	Marks the demo account (2,000 `seed://` invoices) in production (H9)
New table `AnalysisJob`	table	userId, type, state, result (json), costUsd, timestamps; latest per user and type is reused (H1, H10)
New table `ExtractionJob`	table	invoiceId, state (queued, running, done, failed), attempts, runAfter, lastError, timestamps. A Postgres-backed queue is enough below 200 invoices a day (A5)

> **Not stored:** the raw model response (E17). The original file stays in RustFS and can be re-extracted at any time.

6 API reference

Base URL `https://invoi.jxtechstudio.com/api`. All invoice endpoints need `Authorization: Bearer <token>` and only ever see the caller's own invoices; another user's invoice returns 404. Request bodies are validated strictly: unknown fields are rejected with 400.

Method	Path	Auth	Purpose
POST	/auth/register	—	Create an account; returns the user, not a token
POST	/auth/login	—	Returns `{ access_token }`, valid 1 day
GET	/auth/me	JWT	Current user's profile
POST	/invoices/upload	JWT	Multipart field `file`; returns `{ invoice_id }`
GET	/invoices	JWT	List, newest first; filters `status`, `date`, `search`
GET	/invoices/export	JWT	Same filters; CSV download
GET	/invoices/:id	JWT	One invoice
GET	/invoices/:id/status	JWT	`{ invoiceId, status }` for polling
GET	/invoices/:id/file new	JWT	Streams the original file inline (pdf, jpeg, png); owner only; 404 when there is no file
PUT	/invoices/:id	JWT	Edit fields; re-runs the review rules
POST	/invoices/:id/approve new	JWT	Human sign-off; sets `completed`
POST	/invoices/:id/flag new	JWT	Report an error; sets `needs_review` + `user_flagged`
DELETE	/invoices/:id	JWT	Deletes the row and its stored file, if any; 204
GET	/health, /metrics	—	`{ status: "ok" }`; Prometheus text format

Planned

Method	Path	Purpose
POST	/invoices/:id/extract	Re-extract one invoice; never overwrites `editedFields`; 409 while a job is running; counts toward the budget
PATCH	/auth/me	Update `businessNameEn` and `reviewAmountThreshold` from the settings page
POST	/analytics	`{ type }` → `{ jobId }`; daily limit 5 (H1, H10)
GET	/analytics/jobs/:id, /analytics/latest	Poll one job; latest result per type for the page
POST	/internal/n8n/results	n8n callback, `X-Invoi-Secret` only; analysis results and anomaly verdicts (H8)
PUT, POST	/invoices/:id, approve, flag	Return 409 while the invoice is `processing` with an active job (D15)

Rules worth knowing

- **Upload:** PDF, JPEG or PNG, up to 10 MB (`MAX_UPLOAD_SIZE_BYTES`). One file per request. The invoice is created with status `processing` and no extracted fields.
- **Search** matches vendor name, invoice number and customer name, case-insensitive.

- **Edit (PUT):** any of `vendorName, invoiceNumber, invoiceDate, dueDate, totalAmount, taxAmount, amountPaid, currency, paymentStatus, paymentMethod, customerName, taxNumber, crNumber`; at least one field. Amounts are non-negative with up to 2 decimals. Sending `null` clears a field. `status` cannot be set directly. The frontend form sends only changed fields and checks due date ≥ invoice date.
- **Approve:** 400 unless `vendorName`, `totalAmount` and `invoiceDate` are set; then `completed` and all reasons cleared, even if rules had fired. A total of zero is allowed.
- **Flag:** adds `user_flagged`, which survives edits and is cleared only by approval. An invoice in `needs_review` stays there after its flags are fixed until someone approves it.
- **File:** Content-Type comes from the stored extension, never from stored metadata; served inline with `nosniff` and `no-store`. Local storage refuses symlinks and directories. Demo invoices with `seed://` references have no stored file: the file returns 404 and delete removes only the row (PR #113). The frontend fetches the file with the token as a Blob.
- **CSV export:** UTF-8 with BOM so Excel shows Arabic correctly. Cells starting with `= + - @` are prefixed with `'` to block formula injection.
- **Register validation:** password 8+ characters with upper, lower, digit and symbol; phone `+9665` + 8 digits; names in Arabic or English letters; email unique (409 if taken).
- **Rate limits:** 60 requests per minute per IP, 10 per minute on login and register. Express trusts exactly one proxy hop, so `req.ip` is the address Nginx saw and a forged `X-Forwarded-For` no longer resets the login budget.
- **Errors** come back as JSON through a global exception filter, which logs every 5xx with its stack. A 401 makes the frontend clear the token and send the user to `/auth/login?reason=session_expired`. Logout clears the React Query cache and in-memory notifications; login clears the cache.

> **Endpoints the frontend still expects:** forgot/reset password (the page now says the feature is not available yet), settings (English business name, `reviewAmountThreshold`), and JSON dashboard metrics (the dashboard calls `GET /metrics`, which returns Prometheus text).

7 Frontend

The app is right-to-left Arabic by default, with a feature-based folder per area. Pages are lazy-loaded; everything except the landing and auth pages sits behind `ProtectedRoute`, which checks for a token in `localStorage` (`authToken`).

Route	Page	State
/	Landing page	Done
/auth/login, /auth/register	Login and sign-up	Done
/auth/forgot-password	Forgot password	UI only: shows "not available yet" instead of a fake success; backend in Phase 2
/dashboard	KPI cards, processing volume, confidence distribution, top vendors, recent invoices	Built from the invoice list; accuracy card waits on a metrics API
/invoices	Invoice table with search (400 ms delay before searching), status and date filters, CSV export of the current filters, view file	Done · Phase 1: poll while any row is processing
/upload-invoice	Drag-and-drop upload	Done
/invoices/:id	File preview (image or PDF), edit form with all 13 fields, approve, report error, review reasons in Arabic, delete with a confirmation dialog	Done · Phase 1: polling, edit lock, re-extract button
/vendors	Vendors aggregated from invoices	Done
/analytics	Buttons that trigger n8n analysis workflows	Placeholder Phase 3
/settings	Account settings	Placeholder Phase 1: English name and threshold

How data flows. Every call goes through one Axios client (`services/api/client.ts`) that adds the Bearer token and handles 401s. Server state is cached with TanStack Query. The base URL is `VITE_API_BASE_URL`, `/api` by default. Arabic labels live in `invoice/constants/invoiceLabels.ts`.

Modals. Edit and delete stay open until the request succeeds, so validation and server errors show inside the modal (`MainModal keepOpenOnConfirm`). After a delete the invoice list reloads.

Notifications. An in-memory MVP (`notificationsContext.tsx`, PR #94): the upload page adds a notification after each successful upload and the bell in the nav bar shows it. Notifications are lost on refresh and cleared on logout. The plan is a notifications table plus an endpoint, swapping only the context file (Phase 3).

Design system

Emerald 700 #047857

Emerald 600 #059669

Emerald 500 #10B981

Slate 900 #0F172A

Slate 600 #475569

Slate 50 #F8FAFC

Success #84CC16

Warning #F59E0B

Error #EF4444

Info #3B82F6

Token	Value
Primary	Emerald 500 `#10B981` (hover `#059669`, active `#047857`)
Status	Success `#84CC16`, warning `#F59E0B`, error `#EF4444`, info `#3B82F6`
Light backgrounds	Layout `#F8FAFC`, container `#FFFFFF`, border `#E2E8F0`
Dark backgrounds	Layout `#0F172A`, container `#1E293B`, border `#334155`
Fonts	IBM Plex Sans Arabic (headings and body), Inter (Latin UI)
Shape	Border radius 8 px (cards 12 px), control height 40 px
Layout	12-column grid, 1440 px max width, 8 px spacing scale

Tokens live in `src/index.css` (CSS variables) and `src/config/theme/theme.ts` (antd). Use `token.colorPrimary` and the CSS variables instead of hard-coded hex values so dark mode keeps working.

8 AI extraction: Phase 1 design

The review rules are built; the extraction that should feed them is not. This section is the agreed design from the 21 Oct 2026 decision round, with the 22 Oct amendments. Section 9 lists each decision with its code (A1, B6, ...) so it can be traced from here.

The 12 fields to extract: vendor name, invoice number, invoice date, due date, total amount, tax amount, currency, payment status, payment method, customer name (as printed), vendor tax number, vendor CR number. Line items are out of scope.

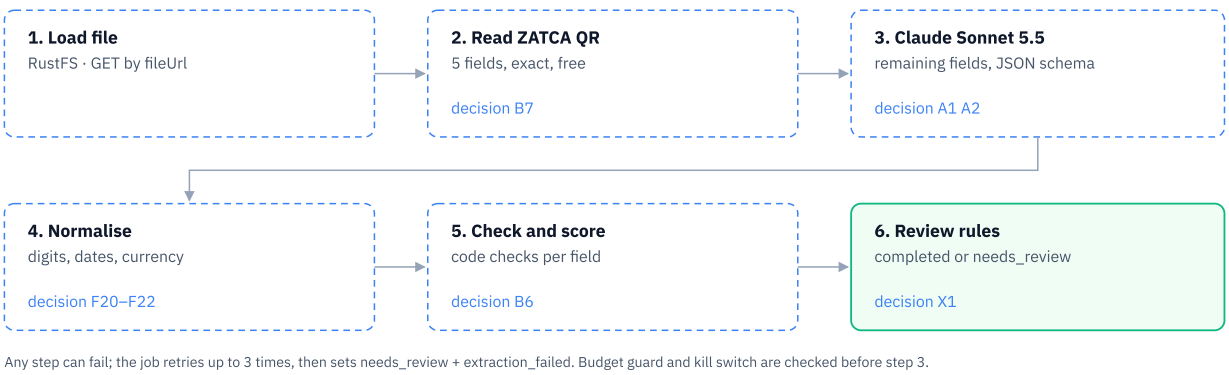


Figure 3 · extraction pipeline, one job per uploaded invoice

How it runs

1. **Queue.** Upload creates the invoice and an `ExtractionJob` row in the same transaction. A worker inside the NestJS process claims jobs with `SELECT … FOR UPDATE SKIP LOCKED`, one or two at a time. No Redis: under 200 invoices a day this is enough (A5, C10).
2. **Kill switch and budget.** Before each call the worker checks `EXTRACTION_ENABLED`, the account allow-list and the month's spend (sum of `extractionCostUsd`). At the cap it stops, leaves jobs queued and logs an alert; nothing is lost (A4, G25).
3. **ZATCA QR first.** Decode the QR (TLV, Base64) from the image, or from rendered PDF pages. Tags 1–5 give seller name, VAT number, timestamp, total with VAT and VAT total. Decoded values get score 1.0 once they pass the checks (B7).
4. **Model for the rest.** One call to Claude Sonnet 5.5 with the file and a fixed JSON schema for all 12 fields; QR values are passed in as known. Payment status and method only when clearly printed; anything not on the page comes back null (A1, A2, F23).
5. **Normalise.** Arabic-Indic digits and the Arabic decimal separator to ASCII; dates to ISO; currency to the enum. Hijri dates are not converted: the field is left empty with a low score so a human enters it (F20, F21, F22).
6. **Check and score** each field with code (below), then run the review rules. Clean result: `completed`. Any rule fires: `needs_review` with reasons (B6, X1).

Confidence: checks done by code

Each field starts at 1.0 and loses score for every failed check; below 0.90 triggers `low_confidence`. The model's own view is not trusted as a score (B6).

Field	Check	On failure
taxNumber	15 digits, starts and ends with 3; equals QR tag 2 when both exist	score 0.5
taxAmount, totalAmount 22 Oct	Only for SAR invoices whose vendor has a valid Saudi VAT number (one that passes the taxNumber check): VAT ≈ 15% of (total − VAT) or exactly 0, ±1 SAR or 1%; equal to QR tags 4–5 when present. Other currencies, or no valid Saudi VAT number: check skipped, no penalty	score 0.6 on both
totalAmount	Greater than zero (also its own rule below)	score 0.5
amountPaid	Not above the total	score 0.6
invoiceDate, dueDate	Valid Gregorian date, not in the future, after 2000; due date ≥ invoice date; Hijri or unreadable → empty	score 0.5 or empty
currency	One of the 13 enum values; anything else → empty	score 0.5
crNumber	10 digits	score 0.7
Any required field	vendorName, totalAmount, invoiceDate missing	score 0

Review rules

An invoice moves to `needs_review` when any rule fires, and `needsReviewReason` lists every rule that did. Source: `backend/src/invoices/invoice-review.ts`.

Rule	Code	In the code?
Any field's confidence below 0.90	`low_confidence`	Yes
Customer name differs from `businessName` and from `businessNameEn`	`customer_name_mismatch`	Yes English name in Phase 1
Total or tax above the account's `reviewAmountThreshold`	`amount_threshold_exceeded`	Yes settable in Phase 1
User reported an error; kept until approval	`user_flagged`	Yes PR #108
Extraction failed after 3 attempts	`extraction_failed`	Phase 1
Total is zero; can still be approved	`zero_total`	Phase 1
Tax above 30% of the total, on every invoice whatever its currency or VAT number 22 Oct	`tax_over_30_percent`	Phase 1
File holds more than one invoice; first one extracted	`multiple_invoices`	Phase 1

Failure, retries and re-extraction

- **Retries:** 3 attempts with growing gaps (about 1 min, 5 min, 15 min). Then `needs_review` with `extraction_failed`, and the user fills the fields in by hand (C11). A file that is clearly not an invoice fails at once, without retries.
- **Re-extract button** on the detail page calls `POST /invoices/:id/extract`. Fields in `editedFields` are never overwritten; every PUT adds the fields it changed (C12).
- **Existing invoices:** a one-time backfill queues every invoice still in `processing` with empty fields, within the monthly budget, and only for accounts listed in `EXTRACTION_ACCOUNT_ALLOWLIST`. It does not run while the allow-list is empty, and the demo account is always skipped (C13, amended 22 Oct).

Frontend during extraction

- **Polling:** every 5 seconds on `GET /invoices/:id/status`, only on pages showing a `processing` invoice, stopping after 2 minutes. On the list page one list refetch replaces per-row polling. 12 requests a minute stay well inside the 60/min limit (D14).

- **While processing:** the user can view the invoice and its file, but edit, approve and report error are disabled, with a short Arabic note. The backend enforces the same with 409 (D15).
- **New reason labels** in Arabic for `extraction_failed`, `zero_total`, `tax_over_30_percent` and `multiple_invoices` in `invoiceLabels.ts`.
- **Settings page:** English business name and review threshold, via `PATCH /auth/me` (F19).

Data, privacy and cost

- **Outside KSA is allowed on one condition:** zero data retention at the provider. Confirm it in writing with Anthropic for this account before the first real invoice is sent; until then, test only on the anonymised test set (E16).
- **Account:** JX Tech Studio company account. Key `ANTHROPIC_API_KEY` in `/run/secrets/invoi.env`, like `JWT_SECRET` (A3).
- **Stored per invoice:** model, duration and cost. The raw response is not stored (E17).
- **Upload limit** unchanged: 10 MB, one file per upload; multi-file upload later (E18).

> **Budget is the tightest constraint.** The cap is USD 2 a month until the project gets its own budget (A4), shared with the analytics AI (H2). At an estimated USD 0.01 per one-page invoice on Sonnet 5.5 that is about 200 invoices a month, against an expected volume of up to 200 a day. The model comparison (about 50 invoices on Sonnet and 50 on Haiku, roughly USD 0.55) and the backfill also come out of it. Real costs must be measured on the test set; raise the cap before inviting more than one or two accounts.

New environment variables

Variable	Purpose
ANTHROPIC_API_KEY	Required when extraction is enabled
EXTRACTION_ENABLED	`true` / `false`; kill switch; no code change, only an app restart
EXTRACTION_ACCOUNT_ALLOWLIST	Comma-separated user IDs during gradual rollout; empty means everyone for new uploads (the backfill never runs while it is empty)
EXTRACTION_MODEL	Default `claude-sonnet-5-5`
EXTRACTION_MONTHLY_BUDGET_USD	Default `2`; processing stops at the cap

Measuring accuracy and rolling out

- **Test set:** Jana prepares 30–50 real invoices with their correct values: Arabic and English, PDFs and phone photos, some poor scans, some with Arabic-Indic digits, with sensitive data removed (B9).
- **Comparison:** a script runs the set on Sonnet 5.5 and Haiku 5.5 and reports field accuracy, cost and time per invoice. Proposed target: 95% of fields correct on clear PDFs. Accuracy is still not a hard MVP requirement.
- **Rollout:** one or two accounts first through the allow-list; the team checks the first ~100 extracted invoices before wider release (G25, G26).

Definition of Done for the MVP: an upload moves through extraction to `completed` or a reviewable state, all 12 fields are shown, the flags fire per the rules above, and the user can correct and save any field.

9 Decision log

Decided by Jana on 21 Oct 2026; B6, B8 and C13 amended on 22 Oct (highlighted). Extraction decisions use codes A–G and X; analytics and n8n decisions use H. "Default" means the proposed option was kept without changes.

#	Question	Decision
A1	Kind of service	General document-reading AI model, one call for all fields
A2	Which model	Claude Sonnet 5.5; measured against Haiku 5.5 on the test set before committing
A3	Account and key	JX Tech Studio company account; key in `/run/secrets/invoi.env`
A4	Monthly budget	USD 2 hard cap for now; processing stops and alerts at the cap. To be raised when the project has a budget
A5	Volume	Under 200 invoices a day at launch: Postgres-backed job table, no Redis
B6	Confidence scores	Computed by code checks (VAT format, VAT rate, sums, dates); model view only a hint. **22 Oct:** the 15% check runs only for SAR invoices from a vendor with a valid Saudi VAT number, and accepts ≈15% or exactly 0 (anything else 0.6); otherwise the check is skipped with no penalty
B7	Read ZATCA QR first	Yes; the model fills only what the QR lacks
B8	30% tax and zero-total checks	Both added with extraction; a zero total goes to review but can be approved. **22 Oct:** the 30% rule applies to all invoices; no tax-category field for now
B9	Test set	Jana prepares 30–50 invoices with correct answers. Target proposed: 95% on clear PDFs
C10	Where extraction runs	Background job inside NestJS; n8n stays for alerts and digests
C11	When extraction fails	3 retries with growing gaps, then `needs_review` + `extraction_failed`
C12	Re-extract button	Yes; never overwrites fields the user edited (`editedFields`)
C13	Invoices already uploaded	Processed once (backfill), within the budget. **22 Oct:** allow-listed accounts only, never while the allow-list is empty; demo account always excluded
D14	How the frontend learns	Polling every 5 s, only for processing invoices, stop after 2 min; notifications later
D15	Editing during extraction	Default: view only until it finishes
E16	Sending invoices outside KSA	Allowed only with zero data retention at the provider
E17	What to store	Default: model, timing and cost; no raw response
E18	Upload limit	Default: 10 MB, one file; multi-file later
F19	Arabic vs English business name	English name stored on the account; compare against both
F20	Hijri dates	Not converted; left empty for review
F21	Arabic-Indic digits	Normalised by code; included in the test set
F22	Other currencies	Add EUR, GBP, KWD, QAR, BHD, OMR, EGP, JOD, CNY, INR
F23	Payment status and method	Default: only when clearly printed
F24	Files that aren't one invoice	Default: first invoice extracted, rest flagged; credit notes later
G25	Rollout	Gradual, behind a kill switch and an account allow-list
G26	Who checks results	The team reviews the first ~100 extracted invoices
X1	Status after a clean extraction	`completed` automatically; the user can still report an error

Still open: written confirmation of zero data retention for the account (blocks real invoices); the real monthly budget (blocks rollout beyond one or two accounts); the accuracy target, once the test set has numbers.

Analytics, n8n and Power BI

#	Question	Decision
H1	How a 20–60 s analysis returns	Job and polling: `POST /analytics` returns `jobId`, page polls every 3 s
H2	AI provider and cost in n8n	Same Anthropic account and the same monthly cap as extraction; zero data retention
H3	Free-text question box	Preset buttons only for now
H4	The five scheduled workflows	Run per user instead of posting all companies' data to a team channel
H5	How scheduled results reach the user	Inside the app only, as the latest result on the analytics page
H6	Isolating each company's data	Backend sends the user's data in the webhook; n8n has no database access
H7	Payment date for DPO	New `Invoice.paidAt`; demo data copied once from `updatedAt`
H8	How the anomaly detector flags	Through an internal endpoint with the shared secret; backend adds the reason
H9	Demo account in production	Kept, marked `isDemo`, excluded from extraction and scheduled runs
H10	Limits and history	5 analyses per user per day; latest result per type kept and reused within an hour
H11	Role of Power BI	Empty templates sold with a paid plan; users download and load their own data (CSV)

Still open for analytics: the final list of analysis types and their names (Abdullah); plans and billing for the Power BI tier; the full Confluence page, which could not be read for this version.

10 Analytics, n8n and Power BI

Design agreed on 21 Oct 2026 (decisions H1–H11 in section 9). n8n does the analysis and the AI writing; the backend owns identity, data and budget. **n8n no longer connects to PostgreSQL**, and **Power BI no longer reads the production database**. Owners: Danah (analytics page, frontend and backend), Abdullah (n8n workflows, Power BI templates).

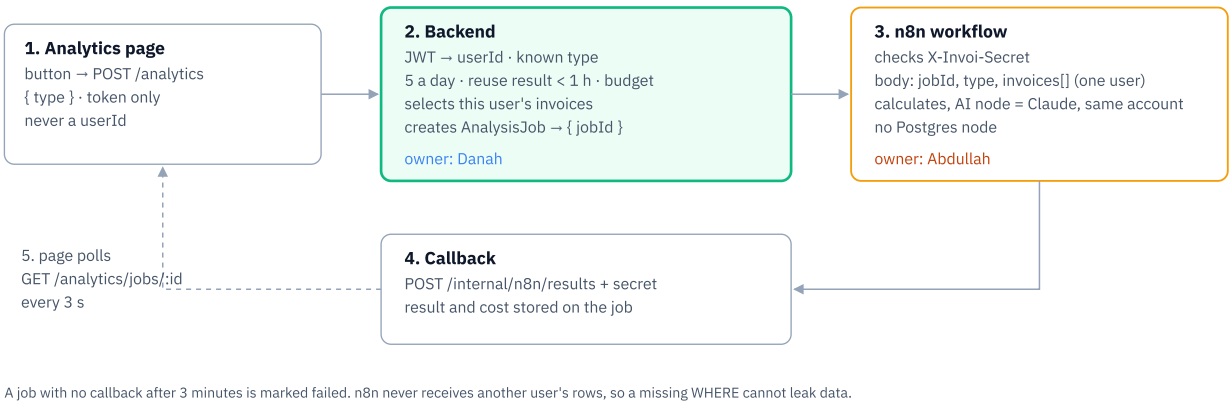


Figure 4 · analysis request flow (job and polling)

Rules

- **Endpoint:** `POST /analytics`. The two team guides disagree (`/analytics` vs `/invoices/analysis`); this document uses the former, as its own Nest module.
- **Request:** preset buttons only; the free-text question waits until isolation and cost are proven (H3). The button list comes from Abdullah's workflows.
- **Data sent:** only fields the analysis needs, for invoices not in `processing`: vendor, number, dates, `paidAt`, amounts, currency, payment status, status (H6).
- **Limits and cache:** 5 analyses per user per day; the same type within an hour returns the stored result at no cost; the latest result per type is shown when the page opens (H10).
- **Cost:** the AI node uses Claude through the company Anthropic account, with zero data retention, inside the same monthly cap as extraction. n8n returns `costUsd` with each result (H2).
- **Currencies:** with 13 currencies, sums are grouped by currency, never added across them.
- **States on the page:** empty, running (button disabled, 20–60 s expected), result, error with "try again".

Result contract (every workflow returns this shape)

```
{
  "jobId": "clx…",
  "type": "dpo_aging",
  "title": "DPO وأعمار المستحقات",
  "generatedAt": "2026-10-21T09:30:00Z",
  "kpis":   [ { "label": "DPO", "value": 43, "unit": "days" } ],
  "chart":  { "kind": "bar",
              "items": [ { "label": "90+", "value": 755720.48, "currency": "SAR" } ] },
  "report": "## الملخص\n…",          // optional Markdown from the AI
  "costUsd": 0.004                  // AI spend for this run, 0 if none
}
```

Workflows after the redesign

Workflow	Trigger	What changes
Payables aging and DPO	Button; every 2 weeks per user	DPO from `paidAt`, not `updatedAt` (H7); remaining = total – amount paid
Cash outflow predictor	Button; weekly per user	Unpaid invoices due in the next 28 days, 4 weekly buckets
Executive narrative	Button; monthly per user	KPIs and period change, written by the AI
Anomaly and duplicate detector	After each extraction	Backend sends the new invoice and that vendor's 6-month history; n8n calls back a verdict; backend adds `duplicate_suspected` or `amount_anomaly` and the invoice goes to `needs_review` (H8)
Financial analyst agent	—	Deferred with the free-text question (H3)

Scheduled runs are per user, not to a team channel (H4). A backend cron starts them for users active in the last 30 days; the result becomes that type's latest result on the analytics page. No email for now; it can come with persistent notifications (H5). While the budget is USD 2 a month, scheduled runs use the extraction allow-list, or they would spend the cap in days.

Demo account in production

The account with 2,000 fake invoices stays in production and is marked with `User.isDemo` (H9). It is excluded from extraction, backfill and scheduled runs, but buttons work on it for testing. Its `seed://` file references have no stored file: `GET /invoices/:id/file` returns 404 and `DELETE` removes the row only (fixed in PR #113; it used to fail with 500).

Power BI: templates sold with a paid plan

Power BI stops reading the database (H11). Each report becomes an empty template (`.pbit`) that users on a plan that includes Power BI can download; they load their own data, usually the CSV export. This needs:

- The CSV export to carry every column the templates use. Today it lacks `amountPaid`, `paidAt`, `status` and the upload date, so unpaid ratio, aging and DPO cannot be computed from it.
- A plan on the account and a download endpoint gated by it. Plans and billing do not exist yet.
- Templates on the brand colours (`#10B981`, `#047857`, `#0F172A`) and Arabic labels.

> **Remove direct database access** once the webhooks are live: the Postgres accounts used by n8n and Power BI (such as `powerbi_reader`) can see every company's invoices.

11 Infrastructure and deployment

invoi runs as three Docker Compose services on one server behind the host's Nginx, which terminates SSL for `invoi.jxtechstudio.com` and proxies to the app on `127.0.0.1:3016`. Every port is bound to localhost; only Nginx is public. The app trusts exactly one proxy hop, so Nginx must stay the only proxy in front of it.

Service	Image	Host port	Volume
app (`invoice-app`)	Built from `Dockerfile` , Node 22 alpine, runs as `node`	127.0.0.1:3016 → 3000	`invoiproject_invoice_uploads` (legacy files)
postgres (`invoice-postgres`)	postgres:16	127.0.0.1:5433 → 5432	`invoiproject_postgres_data`
rustfs (`invoice-rustfs`)	rustfs/rustfs:1.0.0-rc.6	127.0.0.1:9001 (console)	`invoiproject_rustfs_data`

The app waits for healthy Postgres and RustFS, then runs `prisma migrate deploy` and starts. Migrations therefore apply automatically on every deploy, including the Phase 1 migration. The extraction worker runs in the app container and needs outbound HTTPS to the Anthropic API.

Jenkins pipeline (`Jenkinsfile` , one build at a time, env file at `/run/secrets/invoi.env`)

1. **Build** — validate the Compose file, build the app image.
2. **Lint** — ESLint on the backend.
3. **Test** — `npm run test:e2e` inside the app image (15 Jest suites: lifecycle, ownership, review, approval, CORS, storage, metrics, proxy rate limit, exception filter). Extraction tests must mock the model call: CI never spends budget.
4. **Deploy** — `docker compose up -d --build app postgres` , check both containers run, then call `/api/health` up to 5 times.

The frontend has no lint or test stage yet.

Environment variables (`.env.example`)

Variable	Purpose
POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB	Database credentials; Compose builds `DATABASE_URL` from them
JWT_SECRET	Required; signs login tokens
JWT_EXPIRES_IN	In the example file but unused; expiry is hard-coded to 1 day
PORT, NODE_ENV, APP_URL	Runtime basics (port 3000 in the container)
VITE_API_BASE_URL	Frontend API base, baked in at build time; default `/api`
CORS_ALLOWED_ORIGINS	Comma-separated origins; empty means no cross-origin access
STORAGE_DRIVER, STORAGE_LOCAL_PATH	`rustfs` (default) or local disk
RUSTFS_ENDPOINT, RUSTFS_REGION, RUSTFS_BUCKET	Object storage; bucket `invoices`
RUSTFS_ACCESS_KEY, RUSTFS_SECRET_KEY	Required
MAX_UPLOAD_SIZE_BYTES	Upload limit; default 10485760 (10 MB)
ANTHROPIC_API_KEY, EXTRACTION_*	Phase 1 see section 8

Variable	Purpose
N8N_WEBHOOK_BASE_URL, N8N_SHARED_SECRET, ANALYTICS_DAILY_LIMIT	Planned n8n base URL (backend only), the `X-Invoi-Secret` value both ways, analyses per user per day (5)

Monitoring. `/api/metrics` exposes `invoi_http_requests_total` (by method, route and status code) plus Node.js defaults for Prometheus and Grafana. It is public: restrict it to the Prometheus host in Nginx. Unexpected errors reach the container log with their stack. Phase 1 adds extraction counters (jobs by outcome, spend this month).

> **Not in place yet:** scheduled backups of the Postgres and RustFS volumes, a written restore procedure, and the Nginx config in version control.

12 Developer guide

A new developer can run invoi locally in about 15 minutes with Node 22 and Docker.

Run it locally

```
git clone https://github.com/JxTechStudio-org/invoi.git && cd invoi
git checkout develop
cp .env.example .env              # fill in passwords, JWT_SECRET and RustFS keys

# Database and storage only
docker compose up -d postgres rustfs

# Backend on http://localhost:3000/api
cd backend && npm ci
# DATABASE_URL=postgresql://<user>:<pass>@localhost:5433/<db>
# RUSTFS_ENDPOINT=http://localhost:9000 (map port 9000 in Compose for local use)
# EXTRACTION_ENABLED=false unless you are working on extraction
npx prisma generate && npm run prisma:migrate:deploy
npm run start:dev

# Frontend on http://localhost:5173
cd ../frontend && npm install
VITE_API_BASE_URL=http://localhost:3000/api npm run dev
```

Or run the whole stack exactly as production: `docker compose up -d --build`, then open `http://127.0.0.1:3016`.

Useful scripts

Where	Command	Does
backend	`npm run start:dev`	API with watch mode
backend	`npm run lint`	ESLint (CI runs this)
backend	`npm run test:e2e`	Jest e2e suites (CI runs this; needs a database)
backend	`npm run prisma:validate`	Check the schema
frontend	`npm run dev` / `build` / `lint`	Vite dev server, production build, ESLint
docs	`python docs/technical-documentation/build.py`	Export this document to PDF (Chrome or Edge)

Git workflow

- `main` is production; Jenkins deploys it. `develop` is the integration branch.
- Branch from `develop` as `feature/<name>`, `fix/<name>` or `docs/<name>`, and open the PR into `develop`.
- `develop` is merged into `main` by PR once it is green. As of this version the two are in sync (PR #114, 22 Oct).
- Schema changes always ship as a new Prisma migration; never edit an applied one.

Conventions

- Backend: one Nest module per domain; validate every input with a DTO; scope every invoice query by `userId`.
- Frontend: code goes in `features/<area>/{api,components,hooks,pages,constants}`; shared UI in `features/shared`; colours from theme tokens, never hex.
- Text in the UI is Arabic and right-to-left; code, commits and PR titles are in English.
- Never call the paid model from tests or CI; mock the client.
- The repository has 41 remote branches; delete a branch once its PR is merged.
- This document lives in `docs/technical-documentation/`; change it in the same PR as the code it describes.

13 Status, known gaps and roadmap

The platform around extraction is in place and on the production branch: file preview, the approve/complete flow, the list filters and the demo-file fix all landed by 22 Oct. The MVP is now blocked mainly on AI extraction itself.

Closed since v1.0

Gap	What was wrong	Closed by
2	No endpoint to view or download an invoice's file	PR #109 · `GET /invoices/:id/file`, preview and "view file"
3	Nothing sets status to `completed`	PR #108 · approve and report-error
—	Manual edit always failed with 400; only 4 of 13 fields	PR #107 · `InvoiceEditForm`
—	Login rate limit bypassable with `X-Forwarded-For`; 5xx not logged; data of the previous user kept after logout	PR #110
—	No date filter or CSV export on the list; delete without confirmation; edit errors hidden by a closing modal; forgot password showed a fake success	PR #111
5	`seed://` demo invoices: file view and delete returned 500	PR #113 · file 404, delete removes the row only
7	`develop` ahead of `main`	PR #114 · develop merged to main, 22 Oct

Open gaps, most important first

Numbers are kept from v1.2 so references stay valid; closed gaps are listed above.

#	Gap	Area	Plan
1	No AI extraction	Backend	Phase 1 (section 8)
2	30% tax and zero-total checks not coded	Backend	Phase 1, with extraction
3	Edit, approve and flag allowed while `processing`	Both	Phase 1, 409 + disabled buttons
4	Settings page and endpoint missing (threshold, English name)	Both	Phase 1
6	n8n and Power BI hold Postgres accounts that see every company	Infra	Remove when webhooks are live
8	No backups of Postgres or RustFS volumes	Infra	Phase 2
9	`/api/metrics` is public	Infra	Phase 2, Nginx allow-list
10	Dashboard calls `GET /metrics` expecting JSON	Both	Phase 2, JSON stats endpoint
11	Forgot password has no backend	Both	Phase 2, needs an email sender
12	No frontend lint or tests in CI	Infra	Phase 2
13	`JWT_EXPIRES_IN` ignored; expiry fixed at 1 day	Backend	Phase 2
14	Token stored in `localStorage`	Frontend	Later; httpOnly cookie
15	Notifications live in memory only	Both	Phase 3
16	CSV export lacks amount paid, paid date, status and upload date	Backend	Before Power BI templates
17	Root README points to a missing `reports/`; frontend README is the Vite template	Docs	Any time

Phase 1 work, in order

#	Task	Covers
1	Schema migration	Currencies, `businessNameEn`, `editedFields`, extraction metadata, `ExtractionJob`
2	Test set and correct answers (Jana)	30–50 invoices; can start now, in parallel
3	Job runner	Claiming, 3 retries with backoff, kill switch, allow-list, budget guard, metrics
4	ZATCA QR reader	Image and PDF pages, TLV decode, unit tests with sample QRs
5	Model client	Sonnet 5.5, JSON schema output, cost and timing, mocked in tests
6	Normalisation, checks and scores	Digits, dates, currency; checks table (SAR-only VAT check); new review reasons
7	Locks and edit tracking	409 while processing; PUT records `editedFields`
8	Re-extract endpoint and button	`POST /invoices/:id/extract`
9	Frontend polling and labels	Detail and list pages; Arabic labels for new reasons
10	Settings	`PATCH /auth/me` and the settings page
11	Model comparison, then backfill	Sonnet vs Haiku on the test set; backfill allow-listed accounts only
12	Rollout	Zero data retention confirmed; one or two accounts; review first ~100

Roadmap

PHASE 1

MVP complete

AI extraction with ZATCA QR, code-based confidence, the missing review checks, edit locks, re-extract, settings. Done: file preview, approve and complete, list filters.

PHASE 2

Hardening

Backups and restore runbook, metrics locked down, JSON dashboard stats, password reset, frontend CI, JWT expiry from env.

PHASE 3

Insight

Analytics page and n8n webhooks (section 10, may start in parallel), anomaly detector after extraction, persistent notifications, Power BI templates for a paid plan.

PHASE 4

Later

Outgoing invoices, multi-user accounts, ERP integrations, line items, credit notes, multi-file upload.

14 Related documents and changelog

The Notion pages stay as working notes; where they disagree with this document, the code wins. Known differences: the Notion backend page says authentication is not implemented (it is, since 22 Sep 2026), and the Notion workflow diagram shows AI analysis as optional.

Document	Covers
Invoi — Project Documentation (Notion)	Product scope, fields, review rules, Definition of Done
Invoi Back-end / Database schema v4 (Notion)	Earlier API notes; schema with field notes
Invoi Front-end / Workflow Diagram (Notion)	Design system research and routes; upload-to-database flow
Invoi x n8n / Invoi X PowerBI (Notion)	Workflow and dashboard specs
invoi-Visual-Identity.pdf + invoi-Logo-Assets.zip	Logo, colours, type, favicons, social images

Changelog

Version	Changes
v1.3 · 22 Oct 2026	Rebuilt as an editable source in the repository (`docs/technical-documentation/`) with a PDF build script. Synced with `develop` @ 07f2e22 (PRs #111, #113) and `main` (PR #114). Decisions B6, B8 and C13 amended: the 15% VAT check applies only to SAR invoices from vendors with a valid Saudi VAT number, accepts 0 VAT and is otherwise skipped with no penalty; the 30% rule applies to all invoices; no tax-category field; the backfill runs only for allow-listed accounts. Section 7 updated for PR #111 (search delay, CSV export of current filters, delete confirmation, modals stay open on error, forgot password says "not available yet"). Gaps 5 and 7 closed.
v1.2 · 21 Oct 2026	Section 10 rewritten as the analytics, n8n and Power BI design: job and polling, n8n without database access, per-user scheduled runs, result contract, demo account, Power BI templates. Decisions H1–H11. New gaps for `seed://` files, database access and CSV columns.
v1.1 · 21 Oct 2026	Synced with `develop` @ 93a004c (PRs #107–#110): file endpoint, approve and flag, full edit form, auth hardening. Section 8 rewritten as the Phase 1 extraction design; new section 9, decision log; planned schema, endpoints and environment variables; gaps table split into closed and open; Phase 1 task list.
v1.0 · 15 Oct 2026	First edition, synced with `main` (PR #105). Replaced the separate Notion pages as the single reference.

Brand in one line: the name is always lowercase **invoi**; primary colour Emerald 500 `#10B981` with Emerald 700 `#047857` ; text Slate 900 `#0F172A` ; IBM Plex Sans Arabic for all text.